

סיווג בינארי באמצעות Support Vector Machine (SVM)

Binary Classification using Support Vector Machine

ד"ר יורם סגל

כל הזכויות שמורות - © Dr. Segal Yoram

December 2025

גרסה 1.0 - דצמבר 2025

תוכן העניינים

7	1	הקדמה: הקשר היסטורי וסקירת ספרות
7	1.1	רקע היסטורי: Historical Background
7	1.1.1	חלום הפרספטרון: The Perceptron Dream
7	1.1.2	הביקורת של מינסקי ופאפרט: The Minsky-Papert Critique
7	1.1.3	חורף ה-IA: The AI Winter
7	1.2	עליית ה-MVS: The Rise of SVM
8	1.2.1	הרעיון המרכזי: The Core Idea
8	1.3	שימושים מודרניים: Modern Applications
8	1.3.1	דאטאסטים קטנים: Small Datasets
8	1.3.2	פרשנות וסקיפות: Interpretability
8	1.3.3	ערובות תיאורטיות: Theoretical Guarantees
8	1.4	מבנה הספר: Book Structure
10	2	יסודות האופטימיזציה
10	2.1	מהי בעיית אופטימיזציה?: What is an Optimization Problem?
10	2.1.1	מקסימום מול מינימום: Maximum vs Minimum
11	2.2	תכנות ליניארי: Linear Programming
11	2.2.1	הגדרה פורמלית: Formal Definition
11	2.2.2	דוגמה: בעיית המפעל: Example: The Factory Problem
11	2.2.3	אזור הפתרונות האפשריים: Feasible Region
12	2.3	תכנות ריבועי: Quadratic Programming
12	2.3.1	הגדרה פורמלית: Formal Definition
12	2.3.2	קמירות ויחידות הפתרון: Convexity and Solution Uniqueness
12	2.3.3	הקשר ל-MVS: Connection to SVM
12	2.4	כלים לפתרון: Solution Tools
13	2.4.1	שיטת הסימפלקס: The Simplex Method
13	2.4.2	שיטות נקודה פנימית: Interior Point Methods
13	2.4.3	פותרני PQ: QP Solvers
13	2.5	סיכום: Summary
15	3	הגיאומטריה של ההפרדה
15	3.1	המכפלה הסקלרית ככלי החלטה: Inner Product as Decision Tool
15	3.1.1	הגדרה: Definition
15	3.1.2	פרשנות גיאומטרית: Geometric Interpretation
16	3.1.3	כלל הסימן: The Sign Rule
16	3.2	היפר-מישורים: Hyperplanes
16	3.2.1	מה זה היפר-מישור?: What is a Hyperplane?

17	Shifting the Hyperplane: הזזת ההיפר-מישור:	3.2.2
17	Hyperplanes in High Dimensions: היפר-מישורים במימדים גבוהים:	3.2.3
17	Geometric Margin: השוליים הגיאומטריים:	3.3
17	The Problem: Infinite Hyperplanes: הבעיה: אינסוף היפר-מישורים:	3.3.1
17	The SVM Assumption: MVS של ההנחה של:	3.3.2
18	The Separation Channel: תעלת ההפרדה:	3.3.3
18	Support Vectors: וקטורי התמיכה:	3.4
18	Definition: הגדרה:	3.4.1
18	Why Does This Matter?: למה זה חשוב?:	3.4.2
18	Computing Distance: חישוב המרחק:	3.5
18	Distance from Point to Hyperplane: מרחק מנקודה להיפר-מישור:	3.5.1
18	Connection to Inner Product: הקשר למכפלה הסקלרית:	3.5.2
19	Summary: סיכום:	3.6
20	מנוע האופטימיזציה של MVS	4
20	The Naive Formulation: הניסוח הנאיבי:	4.1
20	What Do We Want?: מה אנחנו רוצים?:	4.1.1
20	The Mathematical Monster: המונסטר המתמטי:	4.1.2
21	Trick 1: Linearizing Constraints: לינאריזציה של האילוצים:	4.2
21	The Sign Comparison Problem: הבעיה עם השוואת סימנים:	4.2.1
21	The Trick: Multiply by Label: הטריק: הכפלה בתווית:	4.2.2
22	Trick 2: Canonical Hyperplane: מישור קנוני:	4.3
22	The Problem: Infinite Representations: הבעיה: אינסוף ייצוגים:	4.3.1
22	The Trick: Setting the Scale: קביעת קנה מידה:	4.3.2
23	Geometric Margin Simplifies: השוליים הגיאומטריים מתפשטים:	4.3.3
24	Trick 3: Converting to Minimization: מעבר למינימום:	4.4
24	The Problem: Maximum of a Fraction: הבעיה: מקסימום של שבר:	4.4.1
24	The Trick: Invert and Square: הטריק: הפוך והעלאה בריבוע:	4.4.2
24	The Final Formulation: הניסוח הסופי:	4.4.3
24	From Monster to QP: Summary: סיכום המסע:	4.5
25	Summary: סיכום:	4.6
26	יישום והרחבות	5
26	Solving QP in Practice: פתרון ה-QP בפועל:	5.1
26	Mapping to quadprog: מיפוי ל-quadprog:	5.1.1
26	Building the Matrices: בניית המטריצות:	5.1.2
27	Python Implementation: מימוש ב-Python:	5.2
27	Using cvxopt: שימוש ב-cvxopt:	5.2.1
28	Using sklearn: שימוש ב-sklearn:	5.2.2

29	When to Use What? :מתי להשתמש במה?	5.2.3
30	Multi-class Classification :סיווג רב-מחלקתי	5.3
30	The Problem :הבעיה	5.3.1
30	One-vs-Rest Approach :גישת One-vs-Rest	5.3.2
30	One-vs-One Approach :גישת One-vs-One	5.3.3
31	Support Vectors and the Dual :וקטורי התמיכה והבעיה הדואלית	5.4
31	Who Are the Support Vectors? :מי הם וקטורי התמיכה?	5.4.1
32	The Dual Problem :הבעיה הדואלית	5.4.2
32	Why Is the Dual Important? :למה הדואל חשוב?	5.4.3
32	Complete Example :דוגמה מלאה	5.5
32	Summary :סיכום	5.6
34	סיכום: העתיד של MVS	6
34	Advantages of SVM :היתרונות של MVS	6.1
34	Convex Optimization :אופטימיזציה קמורה	6.1.1
34	Transparency and Interpretability :שקיפות ופרשנות	6.1.2
34	Working with Small Data :עבודה עם מעט נתונים	6.1.3
35	When to Choose SVM? :מתי לבחור MVS?	6.2
35	Small Datasets :דאטאסטים קטנים	6.2.1
35	Interpretability Requirements :דרישות פרשנות	6.2.2
35	Reproducibility Requirements :דרישות שחזוריות	6.2.3
35	SVM in the Deep Learning Era :MVS בעידן ה-Deep Learning	6.3
35	Hybrid Systems :היברידיים	6.3.1
35	Output Layer :שכבת הפלט	6.3.2
36	Few-Shot Learning :למידה עם מעט תוויות	6.3.3
36	Modern Applications :יישומים מודרניים	6.4
36	Medicine :רפואה	6.4.1
36	Finance :פיננסים	6.4.2
36	Bioinformatics :ביואינפורמטיקה	6.4.3
36	What Did We Learn? :מה למדנו?	6.5
37	Conclusion :לסיים	6.6
38	English References	6.7

רשימת האיורים

רשימת הטבלאות

13	Comparison of Optimization Problem Types	השוואת סוגי בעיות אופטימיזציה:	1
16	 Inner Product Decision Rule	כלל ההחלטה של המכפלה הסקלרית:	2
21	Summary of SVM Mathematical Tricks	סיכום הטריקים המתמטיים של SVM:	3
26	SVM to quadprog Parameter Mapping	מיפוי פרמטרים בין SVM ל-quadprog:	4
30	. . . Python SVM Libraries Comparison	השוואת ספריות Python ל-SVM:	5
31	 Multi-class Classification Strategies	אסטרטגיות סיווג רב-מחלקתי:	6

1 הקדמה: הקשר היסטורי וסקירת ספרות

בשנת 1969 התרחש משהו מוזר בעולם הבינה המלאכותית. שני מדענים מבריקים, Marvin Minsky ו-Seymour Papert, פרסמו ספר [1]. הספר הזה כמעט הרג את כל התחום. מה קרה שם? הם הוכיחו שהפרספטרון – המודל הבסיסי של רשתות עצביות – לא יכול לפתור בעיות פשוטות. אפילו פונקציית XOR הייתה מעבר ליכולתו. החוקרים איבדו תקווה. המימון נעלם. תקופה זו נקראת "חורף ה-AI" [2].

1.1 רקע היסטורי: Historical Background

1.1.1 חלום הפרספטרון: The Perceptron Dream

בשנת 1958, Frank Rosenblatt, מאוניברסיטת קורנל המציא את הפרספטרון. הרעיון היה פשוט ומהפכני. בנה מכונה שמחקה את המוח. תן לה ללמוד מדוגמאות. הפרספטרון עבד. הוא למד לזהות דפוסים. התקשורת התלהבה. הניו יורק טיימס הכריז: "המכונה החדשה תוכל ללכת, לדבר, לראות, לכתוב". אבל הייתה בעיה. הפרספטרון היה מוגבל. הוא יכול היה להפריד רק נתונים שניתן להפריד באמצעות קו ישר. בעיות מורכבות יותר היו מעבר ליכולתו.

1.1.2 הביקורת של מינסקי ופאפרט: The Minsky-Papert Critique

Minsky ו-Papert לא סתם ביקרו. הם הוכיחו מתמטית את המגבלות. הפרספטרון לא יכול לחשב פונקציות טופולוגיות. הוא לא יכול לזהות קישוריות. ההוכחה הייתה אלגנטית ומוחצת. היא לא השאירה מקום לתקווה. או שכך נדמה. האמת הייתה מורכבת יותר. Minsky ו-Papert ידעו שרשתות רב-שכבתיות יכולות לפתור את הבעיות האלו. אבל הם טענו שאי אפשר לאמן אותן ביעילות. זו הייתה השערה, לא הוכחה.

1.1.3 חורף ה-AI: The AI Winter

התוצאה הייתה קטסטרופלית. בין השנים 1970 ל-1985, המימון לרשתות עצביות התייבש. חוקרים עברו לתחומים אחרים. ה-AI הסמלי ניצח. רק בשנות ה-80 החלה התעוררות. Hinton, Rumelhart ו-Williams הציגו את אלגוריתם ה-Backpropagation [3]. הם הראו שאפשר לאמן רשתות עמוקות. אבל המחשבים של אז היו חלשים מדי.

1.2 עליית ה-SVM: The Rise of SVM

בדיוק בתקופה הזו צץ פתרון אלגנטי. Vladimir Vapnik עבד על הרעיון כבר מ-1963 [4]. אבל רק ב-1995, יחד עם Corinna Cortes, הוא פרסם את המאמר שישנה הכל [5]. מה עשה את ה-SVM מיוחד?

ראשית, הוא לא דרש כוח חישוב עצום. שנית, יש לו ערובות תיאורטיות. הוא מובטח למצוא את הפתרון האופטימלי. אין מינימומים מקומיים. אין התכנסות למקום הלא נכון. שלישית, הוא עובד מצוין עם מעט נתונים. בעידן של דאטאסטים קטנים, זה היה יתרון עצום.

1.2.1 הרעיון המרכזי: The Core Idea

הרעיון של SVM פשוט להפתיע. יש לנו נקודות בצבעים שונים. אנחנו רוצים קו שמפריד ביניהן.

אבל יש אינסוף קווים אפשריים. איזה קו הוא הטוב ביותר? התשובה: הקו שהכי רחוק מכל הנקודות. אנחנו מחפשים את ה"תעלה" הרחבה ביותר בין המחלקות. הנקודות שנוגעות בשולי התעלה נקראות "וקטורי תמיכה". הם תומכים בקו ההפרדה. מכאן השם: Support Vector Machine.

1.3 שימושים מודרניים: Modern Applications

היום, בעידן ה-Deep Learning, למה בכלל ללמוד SVM? התשובה בשלושה תחומים:

1.3.1 דאטאסטים קטנים: Small Datasets

רשתות עמוקות צריכות מיליוני דוגמאות. SVM עובד מצוין עם מאות. בתחום הרפואה, לעיתים קרובות אין לנו הרבה נתונים. חולים עם מחלה נדירה? אולי עשרות דוגמאות. SVM יכול לעבוד עם זה [6].

1.3.2 פרשנות ושקיפות: Interpretability

SVM הוא שקוף. אנחנו יכולים להסביר למה הוא החליט מה שהחליט. וקטורי התמיכה מראים לנו אילו דוגמאות היו קריטיות. ברפואה, זה חיוני. רופא צריך להבין למה האלגוריתם קבע אבחנה מסוימת.

1.3.3 ערובות תיאורטיות: Theoretical Guarantees

SVM מבוסס על אופטימיזציה קמורה. תמיד יש פתרון יחיד. תמיד נמצא אותו. אין הפתעות. רשתות עמוקות הן קופסה שחורה. SVM הוא ספר פתוח.

1.4 מבנה הספר: Book Structure

ספר זה מוביל את הקורא במסע מתמטי. נתחיל ביסודות האופטימיזציה. נבין מה זה תכנות ליניארי ותכנות ריבועי.

אחר כך נעבור לגיאומטריה. נבין מה עושה מכפלה סקלרית. נלמד איך היפר-מישור מפריד בין מחלקות.

בפרק השלישי נראה את הקסם המתמטי. שלושה טריקים שהופכים בעיה בלתי אפשרית לפתירה.

בפרק האחרון נעבור ליישום. קוד Python. ספריות מוכנות. דוגמאות מעשיות.

בסוף, נסתכל קדימה. איפה SVM מתאים היום? מה עתידו?

המסע מתחיל עכשיו. נפתח בשאלה פשוטה: מה זה בכלל אופטימיזציה?

2 יסודות האופטימיזציה

דמיינו מנהל מפעל. יש לו משאבים מוגבלים: ברזל, זמן תנור, עובדים. הוא רוצה למקסם רווח. מה עליו לייצר?

זו בעיית אופטימיזציה. כל אחד פותר בעיות כאלה בכל יום. לאן לנסוע כדי להגיע הכי מהר? מה לקנות כדי לחסוך הכי הרבה? איך לחלק את הזמן בין משימות?

המתמטיקה נותנת לנו כלים לפתור בעיות כאלה באופן שיטתי. בפרק זה נבנה את הבסיס. נתחיל בפרשט ונתקדם למורכב.

2.1 מהי בעיית אופטימיזציה?: What is an Optimization Problem?

בעיית אופטימיזציה מורכבת משני חלקים.

הראשון: **פונקציית המטרה** (Objective Function). זו הפונקציה שאנחנו רוצים למקסם או למזער. רווח, עלות, מרחק, זמן.

השני: **האילוצים** (Constraints). אלו המגבלות על הפתרון. תקציב מוגבל, זמן מוגבל, משאבים מוגבלים.

הגדרה: בעיית אופטימיזציה

בעיית אופטימיזציה היא מציאת ערכי משתנים x שממזערים (או ממקסמים) פונקציה $f(x)$ תחת אילוצים $g_i(x) \leq 0$.

ללא אילוצים, הבעיה פשוטה. מחפשים נקודה שבה הנגזרת מתאפסת. אבל עם אילוצים, הכל משתנה. הפתרון האופטימלי עלול להיות על גבול האילוצים.

2.1.1 מקסימום מול מינימום: Maximum vs Minimum

רוב האלגוריתמים מחפשים מינימום. אבל לפעמים אנחנו רוצים מקסימום. הטריק פשוט:

$$(1) \quad \max f(x) = -\min(-f(x))$$

מה שמקסם f , ממזער $-f$. לכן תמיד אפשר להמיר בעיית מקסימום לבעיית מינימום. בהמשך נראה שזה בדיוק מה שנעשה עם SVM.

2.2 תכנות ליניארי: Linear Programming

2.2.1 הגדרה פורמלית: Formal Definition

הגדרה: תכנות ליניארי

בעיית תכנות ליניארי (Linear Programming, LP) היא בעיית אופטימיזציה שבה:

- פונקציית המטרה ליניארית: $c^T x$

- האילוצים ליניאריים: $Ax \leq b$

George Dantzig המציא את שיטת הסימפלקס ב-1947 [7]. השיטה הזו שינתה את העולם. היא מאפשרת לפתור בעיות עם אלפי משתנים. תעשיות שלמות תלויות בה: תחבורה, לוגיסטיקה, מימון.

2.2.2 דוגמה: בעיית המפעל: Example: The Factory Problem

נחזור למנהל המפעל שלנו. הוא מייצר שני מוצרים: סירים (x_1) ומחבתות (x_2). הרווח:

- 3 שקלים לכל סיר

- 5 שקלים לכל מחבת

פונקציית המטרה:

$$(2) \quad \max Z = 3x_1 + 5x_2$$

האילוצים:

$$(3) \quad x_1 + x_2 \leq 4 \quad (\text{כמות ברזל})$$

$$(4) \quad 2x_1 + x_2 \leq 6 \quad (\text{זמן תנור})$$

$$(5) \quad x_1 + 3x_2 \leq 9 \quad (\text{כוח אדם})$$

$$(6) \quad x_1, x_2 \geq 0 \quad (\text{אי-שליליות})$$

2.2.3 Feasible Region אזור הפתרונות האפשריים:

כל אילוף מגדיר חצי-מישור. החיתוך של כל חצי-המישורים הוא **אזור הפתרונות האפשריים**. זהו מצולע קמור (Convex Polygon).

תכונה מרכזית: האופטימום תמיד בקודקוד. למה? כי פונקציה ליניארית על מצולע קמור מגיעה למקסימום בקודקוד. זו התובנה המרכזית של שיטת הסימפלקס.

תובנה מרכזית

באופטימיזציה ליניארית, הפתרון האופטימלי תמיד נמצא בקודקוד של אזור האילוצים.

2.3 תכנות ריבועי: Quadratic Programming

מה קורה כשפונקציית המטרה אינה ליניארית? אם היא ריבועית, יש לנו **תכנות ריבועי** (Quadratic Programming, QP).

2.3.1 הגדרה פורמלית: Formal Definition

הגדרה: תכנות ריבועי	
בעיית תכנות ריבועי היא:	
(7)	$\min_x \frac{1}{2} x^T H x + f^T x$
	תחת אילוצים ליניאריים: $Ax \leq b$
	כאשר H היא מטריצה סימטרית.

מדוע יש $\frac{1}{2}$ בנוסחה? זו מוסכמה נוחה. כשגוזרים, ה-2 מהריבוע מתבטל עם ה- $\frac{1}{2}$. התוצאה נקייה יותר.

2.3.2 קמירות ויחידות הפתרון: Convexity and Solution Uniqueness

אם H היא מטריצה חיובית מוגדרת, הבעיה **קמורה** (Convex). זה אומר:

- יש מינימום יחיד

- כל מינימום מקומי הוא גלובלי

- אין מלכודות

זו הסיבה ש-SVM כל כך אמין. בעיית ה-SVM היא בעיית QP קמורה. תמיד נמצא את הפתרון האופטימלי. אין הפתעות.

2.3.3 הקשר ל-SVM: Connection to SVM

בפרקים הבאים נראה שבעיית ה-SVM היא:

$$(8) \quad \min_w \frac{1}{2} \|w\|^2$$

תחת אילוצים ליניאריים.

זו בעיית QP קלאסית. H היא מטריצת היחידה. f הוא וקטור אפסים. האילוצים מגיעים מדרישות הסיווג.

טבלה 1 מסכמת את ההבדלים בין סוגי בעיות האופטימיזציה.

2.4 כלים לפתרון: Solution Tools

איך פותרים בעיות אלו בפועל?

טבלה 1: השוואת סוגי בעיות אופטימיזציה: Comparison of Optimization Problem Types

Example	אילוצים	פונקציית מטרה	סוג הבעיה
Factory problem	ליניאריים $Ax \leq b$	ליניארית $c^T x$	תכנות ליניארי (LP)
SVM	ליניאריים $Ax \leq b$	ריבועית $\frac{1}{2}x^T Hx$	תכנות ריבועי (QP)
General ML	קמורים	קמורה	אופטימיזציה קמורה

2.4.1 שיטת הסימפלקס: The Simplex Method

Dantzig המציא את שיטת הסימפלקס ב-1947. השיטה עוברת מקודקוד לקודקוד. בכל צעד, היא משפרת את פונקציית המטרה. בסוף היא מגיעה לאופטימום. בתיאוריה, השיטה עלולה לקחת זמן אקספוננציאלי [8]. בפועל, היא מהירה מאוד. זו אחת השיטות הכי משומשות בעולם.

2.4.2 שיטות נקודה פנימית: Interior Point Methods

ב-1984, Narendra Karmarkar הציג שיטה חדשה [9]. במקום ללכת על השפה, השיטה עוברת דרך הפנים. היא מובטחת לסיים בזמן פולינומי. לבעיות גדולות מאוד, שיטות נקודה פנימית עדיפות.

2.4.3 פותרני QP: QP Solvers

לבעיות QP, יש פותרנים מיוחדים. Boyd ו-Vandenberghe כתבו את הספר הקלאסי בנושא [10].

בפרק 4 נראה איך להשתמש בפותרנים אלו ב-Python. נשתמש ב-cvxopt ו-scipy.optimize.

2.5 סיכום: Summary

בפרק זה למדנו:

- בעיית אופטימיזציה = פונקציית מטרה + אילוצים
- תכנות ליניארי: פונקציה ואילוצים ליניאריים
- תכנות ריבועי: פונקציה ריבועית, אילוצים ליניאריים
- SVM הוא בעיית QP קמורה

אבל עדיין לא הסברנו איך SVM קשור לכל זה. בפרק הבא נבנה את הגיאומטריה.
נבין מה עושה מכפלה סקלרית. ואז הכל יתחבר.

3 הגיאומטריה של ההפרדה

דמיינו שאתם עומדים על קו. משמאלכם – נקודות אדומות. מימינכם – נקודות ירוקות. הקו מפריד בין העולמות. אבל איך יודעים מאיזה צד אתם? ואיך בוחרים את הקו הטוב ביותר? התשובה מתחילה במושג פשוט: המכפלה הסקלרית.

3.1 המכפלה הסקלרית ככלי החלטה: Inner Product as Decision Tool

3.1.1 הגדרה: Definition

הגדרה: מכפלה סקלרית

המכפלה הסקלרית (Inner Product או Dot Product) בין שני וקטורים $\mathbf{x}$ ו- $\mathbf{w}$ במרחב n -ממדי מוגדרת:

$$(9) \quad \langle \mathbf{x}, \mathbf{w} \rangle = x_1 w_1 + x_2 w_2 + \cdots + x_n w_n = \sum_{i=1}^n x_i w_i$$

כאשר $\mathbf{x}, \mathbf{w} \in \mathbb{R}^n$.

זה נראה כמו נוסחה יבשה. אבל יש לה משמעות גיאומטרית עמוקה.

3.1.2 פרשנות גיאומטרית: Geometric Interpretation

נניח ש- $\mathbf{w}$ הוא וקטור יחידה (אורכו 1). מה המכפלה $\langle \mathbf{x}, \mathbf{w} \rangle$ נותנת לנו? היא נותנת את ההיטל (Projection) של $\mathbf{x}$ על הכיוון של $\mathbf{w}$. זה המרחק מהראשית לנקודה שבה $\mathbf{x}$ "נופל" על הקו שמוגדר על ידי $\mathbf{w}$.

דוגמה מספרית

נניח:

$$\mathbf{x} = (a, b)$$

$$\mathbf{w} = (1, 0)$$

אז:

$$\langle \mathbf{x}, \mathbf{w} \rangle = a \cdot 1 + b \cdot 0 = a$$

התוצאה היא פשוט הרכיב הראשון של $\mathbf{x}$. זה ההיטל על ציר ה- x .

3.1.3 כלל הסימן: The Sign Rule

הנה התובנה המרכזית:

הסימן של $\langle \mathbf{x}, \mathbf{w} \rangle$ אומר לנו מאיזה צד של הקו $\mathbf{x}$ נמצא.
טבלה 2 מסכמת את כלל ההחלטה.

טבלה 2: כלל ההחלטה של המכפלה הסקלרית: Inner Product Decision Rule

תוצאה	מיקום הנקודה	סימן $\langle \mathbf{x}, \mathbf{w} \rangle$
חיובי (+)	באותו צד של L כמו w	$\langle \mathbf{x}, \mathbf{w} \rangle > 0$
שלילי (-)	בצד הנגדי של L	$\langle \mathbf{x}, \mathbf{w} \rangle < 0$
אפס	על הקו L עצמו	$\langle \mathbf{x}, \mathbf{w} \rangle = 0$

תובנה מרכזית

המכפלה הסקלרית היא כלל החלטה. היא אומרת לנו לאיזו מחלקה שייכת נקודה.
חיובי = צד אחד. שלילי = צד שני. אפס = על הקו עצמו.

3.2 היפר-מישורים: Hyperplanes

3.2.1 מה זה היפר-מישור?: What is a Hyperplane?

במרחב דו-ממדי, קו מפריד את המישור לשני חלקים. במרחב תלת-ממדי, מישור מפריד את המרחב לשני חלקים. במרחב n -ממדי, היפר-מישור מפריד את המרחב לשני חלקים.

הגדרה: היפר-מישור

היפר-מישור במרחב $\mathbb{R}^n$ הוא קבוצת כל הנקודות $\mathbf{x}$ המקיימות:

$$(10) \quad \langle \mathbf{x}, \mathbf{w} \rangle = 0$$

כאשר $\mathbf{w}$ הוא הווקטור הנורמלי להיפר-מישור.

שימו לב: ההיפר-מישור עובר דרך הראשית. כל נקודה על ההיפר-מישור מאונכת ל- $\mathbf{w}$.
לכן המכפלה הסקלרית שלה עם $\mathbf{w}$ היא אפס.

3.2.2 Shifting the Hyperplane: מישור-היפר-הזזת

מה אם נרצה היפר-מישור שלא עובר דרך הראשית? מוסיפים **הסטה** (Bias), שנשמך b :

$$(11) \quad \langle \mathbf{x}, \mathbf{w} \rangle + b = 0$$

כעת כלל ההחלטה הופך ל:

$$(12) \quad h_{\mathbf{w},b}(\mathbf{x}) = \text{sign}(\langle \mathbf{w}, \mathbf{x} \rangle + b)$$

אם התוצאה חיובית, הנקודה מעל ההיפר-מישור. אם שלילית, מתחת. אם אפס, על ההיפר-מישור עצמו.

3.2.3 Hyperplanes in High Dimensions: היפר-מישורים במימדים גבוהים:

במרחב n -ממדי, היפר-מישור הוא יריעה ממימד $n - 1$.

- ב- $\mathbb{R}^2$: קו (ממד 1)

- ב- $\mathbb{R}^3$: מישור (ממד 2)

- ב- $\mathbb{R}^{100}$: יריעה מממד 99

זה נשמע מופשט. אבל המתמטיקה זהה לחלוטין. המכפלה הסקלרית עובדת באותו אופן בכל מימד.

3.3 השוליים הגיאומטריים: Geometric Margin

3.3.1 The Problem: Infinite Hyperplanes: אינסוף היפר-מישורים:

נניח שיש לנו נתונים שניתן להפריד. יש אינסוף היפר-מישורים שמפרידים אותם. איזה לבחור?

3.3.2 The SVM Assumption: ההנחה של SVM:

ההנחה של SVM היא פשוטה:

היפר-מישור שרחוק יותר מכל הנקודות – יכליל טוב יותר. למה? כי הוא פחות רגיש לרעש. נקודה חדשה שקרובה לגבול עדיין תסווג נכון.

הגדרה: שוליים גיאומטריים

השוליים הגיאומטריים (Geometric Margin) של היפר-מישור w ביחס לדאטאסט D הם: המרחק הקצר ביותר מנקודת אימון x_i להיפר-מישור המוגדר על ידי w .

3.3.3 תעלת ההפרדה: The Separation Channel

אפשר לחשוב על השוליים כעל "תעלה". ההיפר-מישור עובר באמצע התעלה. הנקודות הקרובות ביותר נוגעות בדפנות התעלה. המטרה של SVM: למצוא את התעלה הרחבה ביותר.

3.4 וקטורי התמיכה: Support Vectors

3.4.1 הגדרה: Definition

הגדרה: וקטורי תמיכה

וקטורי התמיכה (Support Vectors) הם הנקודות הקרובות ביותר להיפר-מישור ההפרדה. הם "תומכים" בהיפר-מישור ומגדירים את מיקומו.

זו תובנה עמוקה. ההיפר-מישור האופטימלי נקבע רק על ידי מספר קטן של נקודות. כל שאר הנקודות לא משפיעות.

3.4.2 למה זה חשוב?: Why Does This Matter?

יש לכך שתי השלכות:
יעילות חישובית. לא צריך לשמור את כל הדאטאסט. מספיק לשמור את וקטורי התמיכה.
יציבות. שינוי בנקודות רחוקות לא משנה את ההיפר-מישור. רק שינוי בוקטורי התמיכה משפיע.

3.5 חישוב המרחק: Computing Distance

3.5.1 מרחק מנקודה להיפר-מישור: Distance from Point to Hyperplane

המרחק מנקודה x להיפר-מישור $\langle \mathbf{w}, \mathbf{x} \rangle + b = 0$ הוא:

$$(13) \quad \text{distance} = \frac{|\langle \mathbf{w}, \mathbf{x} \rangle + b|}{\|\mathbf{w}\|}$$

כאשר $\|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$ הוא אורך הווקטור $\mathbf{w}$.

3.5.2 הקשר למכפלה הסקלרית: Connection to Inner Product

שימו לב: המונה הוא בדיוק המכפלה הסקלרית (עם ערך מוחלט). המכנה מנרמל לפי אורך $\mathbf{w}$.

אם $\mathbf{w}$ הוא וקטור יחידה ($\|\mathbf{w}\| = 1$), המרחק הוא פשוט:

$$(14) \quad \text{distance} = |\langle \mathbf{w}, \mathbf{x} \rangle + b|$$

3.6 סיכום: Summary

בפרק זה למדנו:

- המכפלה הסקלרית היא כלל החלטה
 - הסימן קובע מאיזה צד הנקודה
 - היפר-מישור מפריד את המרחב לשני חלקים
 - השוליים הם המרחק לנקודה הקרובה ביותר
 - וקטורי התמיכה מגדירים את ההיפר-מישור
- עכשיו יש לנו את כל הכלים הגיאומטריים. אבל איך מוצאים את ההיפר-מישור האופטימלי? בפרק הבא נראה את הקסם המתמטי. שלושה טריקים שהופכים בעיה בלתי אפשרית לבעיית QP פשוטה.

4 מנוע האופטימיזציה של SVM

הבעיה נראית פשוטה. מצא את ההיפר-מישור שממקסם את השוליים. אבל כשמנסים לכתוב את זה מתמטית, מגלים סיט.

יש לנו מינימום בתוך מקסימום. יש לנו השוואות סימנים מסובכות. הבעיה לא ליניארית. איך הופכים את הסיט הזה לבעיית אופטימיזציה פתירה? בשלושה טריקים מבריקים.

4.1 הניסוח הנאיבי: The Naive Formulation

4.1.1 מה אנחנו רוצים?: What Do We Want?

נתחיל מהתחלה. יש לנו דאטאסט של m נקודות: $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$. כל $\mathbf{x}_i \in \mathbb{R}^n$. הוא וקטור מאפיינים. כל $y_i \in \{-1, +1\}$ הוא התווית.

אנחנו מחפשים היפר-מישור $(\mathbf{w}, b)$ שמפריד בין המחלקות. ולא סתם מפריד. מפריד עם שוליים מקסימליים.

4.1.2 המונסטר המתמטי: The Mathematical Monster

השוליים הגיאומטריים הם המרחק מההיפר-מישור לנקודה הקרובה ביותר. בסימון מתמטי:

$$(15) \quad \gamma = \min_{i=1, \dots, m} \frac{|\langle \mathbf{w}, \mathbf{x}_i \rangle + b|}{\|\mathbf{w}\|}$$

אנחנו רוצים למקסם את γ . אבל יש בעיה.

הסימן של $\langle \mathbf{w}, \mathbf{x}_i \rangle + b$ תלוי בצד שבו הנקודה נמצאת. נקודות חיוביות ($y_i = +1$) צריכות לתת ערך חיובי. נקודות שליליות ($y_i = -1$) צריכות לתת ערך שלילי.

הניסוח הנאיבי נראה כך:

$$(16) \quad \max_{\mathbf{w}, b} \left[\min_{i=1, \dots, m} \frac{|\langle \mathbf{w}, \mathbf{x}_i \rangle + b|}{\|\mathbf{w}\|} \right]$$

תחת האילוצים:

$$(17) \quad \text{sign}(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) = y_i \quad \forall i$$

למה זה בעייתי?

הבעיה הזו היא סיוט חישובי:

- יש min בתוך max

- יש ערך מוחלט

- האילוצים כוללים פונקציית sign

- זו בעיה לא קמורה

אנחנו צריכים גישה אחרת. שלושה טריקים יהפכו את המונסטר הזה לבעיית QP נקייה. טבלה 3 מסכמת את שלושת הטריקים ואת תרומתם.

טבלה 3: סיכום הטריקים המתמטיים של SVM: Summary of SVM Mathematical Tricks

מטרה	אחרי	לפני	טריק
ביטול if-else	$y_i(\langle x_i, w \rangle + b) \geq 0$	$\text{sign}(\langle x_i, w \rangle + b) = \text{sign}(y_i)$	טריק 1
נרמול המרחק	$y_i(\langle x_i, w \rangle + b) \geq 1$	≥ 0	טריק 2
מיזעור במקום מיקסום	$\min \frac{1}{2} \ w\ ^2$	$\max \frac{1}{\ w\ }$	טריק 3

4.2 טריק 1: לינאריזציה של האילוצים: Trick 1: Linearizing Constraints

4.2.1 הבעיה עם השוואת סימנים: The Sign Comparison Problem

האילוץ $\text{sign}(\langle w, x_i \rangle + b) = y_i$ אומר:

- אם $y_i = +1$, אז $\langle w, x_i \rangle + b > 0$

- אם $y_i = -1$, אז $\langle w, x_i \rangle + b < 0$

זה אילוץ מסורבל. יש לו שני מקרים. איך מאחדים אותם?

4.2.2 הטריק: הכפלה בתוויית: The Trick: Multiply by Label

הפתרון אלגנטי להפליא. נכפיל את שני הצדדים ב- y_i :

$$(18) \quad y_i \cdot (\langle w, x_i \rangle + b) > 0$$

למה זה עובד?

- אם $y_i = +1$: הביטוי נשאר כמו שהוא, וחיובי כפול חיובי = חיובי

- אם $y_i = -1$: הביטוי מתהפך בסימן, ושילי כפול שלילי = חיובי

בשני המקרים התוצאה חיובית. האילוץ הופך לאילוץ אחיד:

$$(19) \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) > 0 \quad \forall i$$

הגדרה: שוליים פונקציונליים

השוליים הפונקציונליים (Functional Margin) של נקודה $(\mathbf{x}_i, y_i)$ מוגדרים:

$$(20) \quad \hat{\gamma}_i = y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b)$$

ערך זה חיובי אם הנקודה מסווגת נכון.

עכשיו הבעיה נראית יותר נקייה:

$$(21) \quad \max_{\mathbf{w}, b} \left[\min_i \frac{y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b)}{\|\mathbf{w}\|} \right]$$

תחת האילוץ:

$$(22) \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) > 0 \quad \forall i$$

טוב יותר. אבל עדיין יש min בתוך max.

4.3 טריק 2: מישור קנוני: Trick 2: Canonical Hyperplane

4.3.1 הבעיה: אינסוף ייצוגים: The Problem: Infinite Representations

לכל היפר-מישור יש אינסוף ייצוגים. אם $(\mathbf{w}, b)$ מגדיר היפר-מישור, גם $(2\mathbf{w}, 2b)$ מגדיר את אותו היפר-מישור. וגם $(100\mathbf{w}, 100b)$. זו בעיה. אנחנו צריכים ייצוג יחיד.

4.3.2 הטריק: קביעת קנה מידה: The Trick: Setting the Scale

נבחר את קנה המידה כך שהנקודה הקרובה ביותר תהיה במרחק פונקציונלי של בדיוק 1. כלומר:

$$(23) \quad \min_i y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) = 1$$

זה נקרא מישור קנוני (Canonical Hyperplane).

למה זה גאוני? כי עכשיו ה-min נעלם מהבעיה. אנחנו יודעים שהמינימום הוא בדיוק 1.

נניח שיש לנו היפר-מישור עם:

$$\mathbf{w} = (2, 4)$$

$$b = 6$$

והנקודה הקרובה ביותר נותנת $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) = 3$
כדי להפוך לייצוג קנוני, נחלק ב-3:

$$\mathbf{w}' = \left(\frac{2}{3}, \frac{4}{3} \right)$$

$$b' = 2$$

עכשיו הנקודה הקרובה נותנת בדיוק 1.

4.3.3 השוליים הגיאומטריים מתפשטים: Geometric Margin Simplifies

עכשיו השוליים הגיאומטריים הופכים לפשוטים:

$$(24) \quad \gamma = \frac{1}{\|\mathbf{w}\|}$$

למה? כי השוליים הפונקציונליים של הנקודה הקרובה הם 1. והשוליים הגיאומטריים הם השוליים הפונקציונליים מחולקים ב- $\|\mathbf{w}\|$.

הבעיה הופכת ל:

$$(25) \quad \max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|}$$

תחת האילוצים:

$$(26) \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 \quad \forall i$$

שימו לב: האי-שוויון הפך מ- $0 < 1 - \gamma \leq$ זה בגלל הנרמול הקנוני. הנקודות הקרובות ביותר נמצאות בדיוק על הגבול $= 1$. כל השאר רחוקות יותר.

וקטורי תמיכה

הנקודות שמקיימות $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) = 1$ נקראות וקטורי תמיכה. הן "תומכות" בהיפר-מישור. הן הנקודות הקריטיות שקובעות את מיקום ההיפר-מישור.

4.4 טריק 3: מעבר למינימום: Trick 3: Converting to Minimization

4.4.1 הבעיה: מקסימום של שבר: The Problem: Maximum of a Fraction

יש לנו:

$$(27) \quad \max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|}$$

זה עדיין לא נוח. פונקציית מטרה עם שבר מסובכת. ואלגוריתמי אופטימיזציה מעדיפים מינימום על מקסימום.

4.4.2 הטריק: היפוך והעלאה בריבוע: The Trick: Invert and Square

מקסימום של $\frac{1}{\|\mathbf{w}\|}$ שקול למינימום של $\|\mathbf{w}\|$. ומינימום של $\|\mathbf{w}\|$ שקול למינימום של $\|\mathbf{w}\|^2$. למה? כי פונקציית הריבוע היא מונוטונית עבור מספרים אי-שלילים. נוסיף גם מקדם $\frac{1}{2}$ מסיבות טכניות:

$$(28) \quad \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

למה $\frac{1}{2}$? כי כשנגזור, ה-2 מהריבוע יתבטל עם ה- $\frac{1}{2}$. הנגזרת תהיה פשוט $\mathbf{w}$. זו מוסכמה נוחה.

4.4.3 הניסוח הסופי: The Final Formulation

בעיית ה-MVS הפרימלית

בעיית ה-SVM בצורתה הסופית:

$$(29) \quad \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

תחת האילוצים:

$$(30) \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 \quad \forall i = 1, \dots, m$$

זו בעיית QP קלאסית. פונקציית המטרה ריבועית. האילוצים ליניאריים. הבעיה קמורה. תמיד נמצא את הפתרון האופטימלי. אין מינימומים מקומיים. אין התכנסות למקום הלא נכון.

4.5 ממונסטר ל-QP: סיכום המסע: From Monster to QP: Summary

הבה נעקוב אחרי המסע:

התחלנו עם:

$$(31) \quad \max_{\mathbf{w}, b} \left[\min_i \frac{|\langle \mathbf{w}, \mathbf{x}_i \rangle + b|}{\|\mathbf{w}\|} \right]$$

תחת $\text{sign}(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) = y_i$
טריק 1 – הכפלה בתווית:

$$(32) \quad \max_{\mathbf{w}, b} \left[\min_i \frac{y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b)}{\|\mathbf{w}\|} \right]$$

תחת $y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) > 0$
טריק 2 – מישור קנוני ($\min = 1$):

$$(33) \quad \max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|}$$

תחת $y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1$
טריק 3 – היפוך וריבוע:

$$(34) \quad \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

תחת $y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1$

תובנה מרכזית

שלושה טריקים פשוטים הפכו בעיה בלתי אפשרית לבעיית QP סטנדרטית. זה הכוח של מתמטיקה טובה.

4.6 סיכום: Summary

בפרק זה למדנו:

- הניסוח הנאיבי של SVM הוא בעיה קשה

- טריק 1: הכפלה בתווית מאחדת את האילוצים

- טריק 2: מישור קנוני מבטל את ה-min

- טריק 3: היפוך וריבוע נותנים בעיית מינימום

- התוצאה: בעיית QP קמורה עם פתרון יחיד

עכשיו יש לנו בעיה מתמטית נקייה. אבל איך פותרים אותה בפועל? בפרק הבא נראה את היישום. נכתוב קוד Python. נשתמש בפתרני QP מוכנים. ונראה איך הכל עובד על נתונים אמיתיים.

5 יישום והרחבות

תיאוריה יפה. מתמטיקה אלגנטית. אבל האם זה באמת עובד?
 התשובה: כן. יש פותרנים מוכנים. יש ספריות בשפות שונות. צריך רק לדעת איך להשתמש בהם.
 בפרק זה נעבור מתיאוריה למעשה. נכתוב קוד. נראה תוצאות.

5.1 פתרון ה-QP בפועל: Solving QP in Practice

5.1.1 מיפוי ל-quadprog: Mapping to quadprog

בעיית ה-SVM שלנו:

$$(35) \quad \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1$$

פותרני QP מצפים לצורה סטנדרטית:

$$(36) \quad \min_{\mathbf{x}} \frac{1}{2} \mathbf{x}^T H \mathbf{x} + \mathbf{f}^T \mathbf{x} \quad \text{s.t.} \quad A \mathbf{x} \leq \mathbf{b}$$

טבלה 4 מציגה את המיפוי בין הפרמטרים.

טבלה 4: מיפוי פרמטרים בין SVM ל-quadprog: SVM to quadprog Parameter Mapping

quadprog	משמעות ב-SVM	פרמטר
$H = I$	מטריצת יחידה (נורמה)	מטריצה ריבועית
$f = 0$	אין רכיב ליניארי	וקטור ליניארי
$A = -\text{diag}(y) \cdot X$	אילוצי סיווג	מטריצת אילוצים
$b = -1$	מרחק מינימלי 1	וקטור אילוצים

5.1.2 בניית המטריצות: Building the Matrices

נסמן את וקטור המשתנים $\mathbf{z} = (\mathbf{w}, b) \in \mathbb{R}^{n+1}$.

מטריצת ה-Hessian:

$$(37) \quad H = \begin{pmatrix} I_{n \times n} & \mathbf{0} \\ \mathbf{0}^T & 0 \end{pmatrix}$$

כאשר I היא מטריצת היחידה. שימו לב: המקדם של b הוא אפס כי b לא מופיע בפונקציית המטרה.

וקטור $\mathbf{f}$:

$$(38) \quad \mathbf{f} = \mathbf{0}$$

אין איברים ליניאריים בפונקציית המטרה.

מטריצת האילוצים: האילוץ $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1$ שקול ל:

$$(39) \quad -y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \leq -1$$

לכן:

$$(40) \quad A = - \begin{pmatrix} y_1 \mathbf{x}_1^T & y_1 \\ y_2 \mathbf{x}_2^T & y_2 \\ \vdots & \vdots \\ y_m \mathbf{x}_m^T & y_m \end{pmatrix}, \quad \mathbf{b} = -\mathbf{1}$$

5.2 מימוש ב-Python: Python Implementation

5.2.1 שימוש ב-cvxopt: Using cvxopt

הספרייה cvxopt מספקת פותרן QP איכותי [11].

פתרון SVM עם cvxopt – חלק 1: הגדרות

```
import numpy as np
from cvxopt import matrix, solvers

def train_svm_cvxopt(X, y):
    """
    Train linear SVM using cvxopt QP solver.

    Parameters:
    X: array, shape (m, n)
        Training samples
    y: array, shape (m,)
        Labels (+1 or -1)

    Returns:
    w: array, shape (n,)
        Weight vector
    b: float
        Bias term
    """
    m, n = X.shape
```

פתרון SVM עם cvxopt - חלק 2: מימוש

```
# Build H matrix (n+1 x n+1)
H = np.zeros((n + 1, n + 1))
H[:n, :n] = np.eye(n)

# Build f vector
f = np.zeros(n + 1)

# Build constraint matrix A
# -y_i * (w.x_i + b) <= -1
A = np.zeros((m, n + 1))
for i in range(m):
    A[i, :n] = -y[i] * X[i]
    A[i, n] = -y[i]

b_vec = -np.ones(m)

# Convert to cvxopt format
P = matrix(H)
q = matrix(f)
G = matrix(A)
h = matrix(b_vec)

# Solve QP
solvers.options['show_progress'] = False
sol = solvers.qp(P, q, G, h)

# Extract solution
z = np.array(sol['x']).flatten()
w = z[:n]
bias = z[n]

return w, bias
```

5.2.2 שימוש ב-sklearn: Using sklearn

לשימוש מעשי, הספרייה scikit-learn נוחה יותר [12].

```

from sklearn.svm import SVC
import numpy as np

def train_svm_sklearn(X, y, kernel='linear'):
    """
    Train SVM using scikit-learn.

    Parameters:
    -----
    X: array, shape (m, n)
        Training samples
    y: array, shape (m,)
        Labels (+1 or -1)
    kernel: str
        Kernel type ('linear', 'rbf', 'poly')

    Returns:
    -----
    clf: SVC object
        Trained classifier
    """
    clf = SVC(kernel=kernel, C=1e10) # Large C = hard margin
    clf.fit(X, y)
    return clf

# Example usage
X = np.array([[1, 2], [2, 3], [3, 1], [4, 2],
              [5, 5], [6, 6], [7, 5], [8, 6]])
y = np.array([-1, -1, -1, -1, 1, 1, 1, 1])

clf = train_svm_sklearn(X, y)
print(f"Weight vector: {clf.coef_}")
print(f"Bias: {clf.intercept_}")
print(f"Support vectors: {clf.support_vectors_}")

```

טבלה 5 משווה בין הספריות השונות.

5.2.3 מתי להשתמש במה?: When to Use What?

cvxopt – כשרוצים שליטה מלאה. מתאים ללימוד, למחקר, ולמקרים מיוחדים. אפשר לראות את כל הפרטים.

sklearn – לשימוש מעשי. ממומש ב-LIBSVM [13]. מהיר, יציב, מוכן לפרודקשן.

scipy – פשרה. `scipy.optimize.minimize` יכול לפתור QP. פחות מיוחד

מהירות	שימוש מומלץ	אלגוריתם	Library
בינונית	דאטאסטים קטנים, kernels	libsvm (SMO)	sklearn.svm.SVC
מהירה	דאטאסטים גדולים, ליניארי	liblinear	sklearn.svm.LinearSVC
משתנה	בעיות QP כלליות	Interior point	cvxopt / scipy

מ-sklearn, פחות נוח מ-cvxopt.

5.3 סיווג רב-מחלקתי: Multi-class Classification

5.3.1 הבעיה: The Problem

SVM הוא מסווג בינארי. הוא יודע להפריד בין שתי מחלקות. מה עושים כשיש יותר?

5.3.2 גישת One-vs-Rest Approach: One-vs-Rest

נניח שיש K מחלקות. נאמן K מסווגים. מסווג k מפריד בין מחלקה k לכל השאר. בזמן חיזוי:

1. מריצים את כל K המסווגים

2. בוחרים את המחלקה עם הניקוד הגבוה ביותר

5.3.3 גישת One-vs-One Approach: One-vs-One

גישה אחרת: נאמן מסווג לכל זוג מחלקות. יש $\binom{K}{2} = \frac{K(K-1)}{2}$ זוגות. בזמן חיזוי:

1. מריצים את כל המסווגים

2. כל מסווג "מצביע" למחלקה שלו

3. המחלקה עם הכי הרבה הצבעות מנצחת

טבלה 6 משווה בין הגישות.

טבלה 6: אסטרטגיות סיווג רב-מחלקתי: Multi-class Classification Strategies

שיטה	מספר מסווגים	יתרונות	חסרונות
One-vs-Rest	K	מהיר יותר	חוסר איזון במחלקות
One-vs-One	$\frac{K(K-1)}{2}$	מאוזן יותר	זמן אימון ארוך

סיווג רב-מחלקתי עם sklearn

```

from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load Iris dataset (3 classes)
iris = load_iris()
X, y = iris.data, iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Train multi-class SVM (default: one-vs-one)
clf = SVC(kernel='linear', decision_function_shape='ovo')
clf.fit(X_train, y_train)

# Evaluate
accuracy = clf.score(X_test, y_test)
print(f"Accuracy: {accuracy:.2%}")

```

5.4 וקטורי התמיכה והבעיה הדואלית: Support Vectors and the Dual

5.4.1 מי הם וקטורי התמיכה?: Who Are the Support Vectors?

בפתרון האופטימלי, רק חלק מהנקודות "פעילות". אלו הנקודות שנמצאות בדיוק על השוליים:

$$(41) \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) = 1$$

נקודות אלו נקראות וקטורי תמיכה.

ההיפר-מישור נקבע רק על ידי וקטורי התמיכה. כל שאר הנקודות יכולות להיעלם – התוצאה תישאר זהה.

5.4.2 הבעיה הדואלית: The Dual Problem

באמצעות כופלי לגרנז', אפשר לנסח את הבעיה אחרת. זו הבעיה הדואלית:

$$(42) \quad \max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

תחת:

$$\alpha_i \geq 0 \quad \forall i \quad (43)$$

$$\sum_{i=1}^m \alpha_i y_i = 0 \quad (44)$$

המשתנים α_i נקראים **כופלי לגרנז'**. וקטורי התמיכה הם בדיוק הנקודות עם $\alpha_i > 0$.

5.4.3 למה הדואל חשוב?: Why Is the Dual Important?

שני יתרונות מרכזיים:

- יעילות חישובית.** בבעיה הדואלית יש m משתנים (מספר הדוגמאות). בבעיה הפרימלית יש $n + 1$ משתנים (מימד המרחב). כשיש הרבה מאפיינים ומעט דוגמאות, הדואל יעיל יותר.
- טריק הגרעין.** שימו לב: בבעיה הדואלית, הנתונים מופיעים רק דרך מכפלות סקלריות $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$. זה פותח את הדלת ל**שיטות גרעין** (Kernel Methods). אפשר להחליף את המכפלה הסקלרית בפונקציה אחרת. בלי לחשב את ההעתקה למרחב גבוה יותר במפורש. זה נושא לספר אחר. אבל חשוב לדעת שהאפשרות קיימת.

5.5 דוגמה מלאה: Complete Example

5.6 סיכום: Summary

בפרק זה למדנו:

- איך למפות את בעיית ה-SVM לפותרן QP

- שימוש ב-cvxopt לשליטה מלאה

- שימוש ב-sklearn לפרודקשן

- סיווג רב-מחלקתי: One-vs-One ו One-vs-Rest

- וקטורי התמיכה והבעיה הדואלית

עכשיו יש לנו את כל הכלים. מהתיאוריה המתמטית ועד הקוד הפועל. SVM אינו קופסה שחורה. אנחנו מבינים כל שלב.

בפרק הבא נסכם את המסע. נשאל: מתי להשתמש ב-SVM? ומה עתידו בעידן ה-Deep Learning?

6 סיכום: העתיד של SVM

סיימנו מסע ארוך. התחלנו בהיסטוריה. עברנו דרך אופטימיזציה וגיאומטריה. הגענו לקוד פועל.

עכשיו הגיע הזמן לשאול: מה הלאה? בעולם של רשתות עמוקות ו-GPT, מה מקומו של SVM?

התשובה מפתיעה. SVM לא מת. הוא פשוט מצא את הנישה שלו.

6.1 היתרונות של SVM: Advantages of SVM

6.1.1 אופטימיזציה קמורה: Convex Optimization

SVM מבוסס על אופטימיזציה קמורה. זה אומר:

- פתרון יחיד ומובטח
- אין מינימומים מקומיים
- אותו אלגוריתם תמיד יתן אותה תוצאה

רשתות עמוקות שונות. האימון מבוסס על Gradient Descent סטוכסטי. התוצאה תלויה באתחול האקראי. שתי הרצות עלולות לתת תוצאות שונות.

דטרמיניזם

SVM הוא דטרמיניסטי. זה יתרון קריטי במערכות שדורשות שחזוריות. רפואה. פיננסים. מערכות בטיחות.

6.1.2 שקיפות ופרשנות: Transparency and Interpretability

SVM הוא שקוף. אנחנו יודעים בדיוק איך הוא מחליט. וקטור המשקולות w אומר לנו מה חשוב. משקולת גבוהה = מאפיין משפיע. משקולת נמוכה = מאפיין פחות רלוונטי. וקטורי התמיכה מראים לנו את הדוגמאות הקריטיות. אלו הנקודות שקובעות את ההחלטה. אפשר לבדוק אותן ולהבין למה המודל מתנהג כפי שהוא מתנהג. רשתות עמוקות הן קופסה שחורה. יש מיליוני פרמטרים. אף אחד לא באמת יודע למה הן מחליטות מה שהן מחליטות.

6.1.3 עבודה עם מעט נתונים: Working with Small Data

רשתות עמוקות צריכות הרבה נתונים. מאות אלפים. מיליונים. בלי נתונים, הן לא עובדות. SVM שונה. הוא יכול לעבוד עם עשרות או מאות דוגמאות. הוא לא זקוק לדאטאסטים עצומים. בתחומים רבים, אין הרבה נתונים:

- מחלות נדירות - מעט חולים
- תקלות חריגות במכונות - מעט דוגמאות
- תחומים חדשים - עוד לא נאספו נתונים
- במקרים כאלה, SVM עדיף.

6.2 מתי לבחור SVM? :When to Choose SVM?

6.2.1 דאטאסטים קטנים: Small Datasets

הכלל פשוט. אם יש לך פחות מ-10000 דוגמאות, שקול SVM. אם יש פחות מ-1000, SVM כנראה עדיף.

6.2.2 דרישות פרשנות: Interpretability Requirements

בתחומים מסוימים, לא מספיק שהמודל עובד. צריך להסביר למה הוא החליט. רופא צריך להסביר לחולה למה הוא מאבחן מחלה. בנק צריך להסביר למה הוא דחה הלוואה. מערכת משפטית צריכה להסביר את הקריטריונים. SVM מאפשר את זה. רשתות עמוקות לא.

6.2.3 דרישות שחזוריות: Reproducibility Requirements

במערכות קריטיות, התוצאה חייבת להיות זהה בכל הרצה. אסור שהאלגוריתם יתנהג אחרת מחר. SVM מבטיח את זה. אותם נתונים, אותו אימון, אותה תוצאה. תמיד.

6.3 SVM בעידן ה-Deep Learning: SVM in the Deep Learning Era

6.3.1 היברידיים: Hybrid Systems

גישה מודרנית משלבת את שני העולמות. רשת עמוקה מחלצת מאפיינים. היא לוקחת תמונה ומייצרת וקטור. הוקטור הזה נכנס ל-SVM. ה-SVM מבצע את הסיווג הסופי. זה נותן את היתרונות של שני העולמות:

- ייצוג עשיר מהרשת העמוקה
- סיווג שקוף ויציב מ-SVM

6.3.2 שכבת הפלט: Output Layer

גם בתוך רשתות עמוקות, SVM נמצא. הפונקציה Hinge Loss של SVM משמשת כפונקציית הפסד. היא מחליפה את Cross-Entropy הרגיל. מחקרים הראו שזה יכול לשפר ביצועים במקרים מסוימים [14].

6.3.3 למידה עם מעט תוויות: Few-Shot Learning

בלמידה עם מעט תוויות, יש רק כמה דוגמאות ממחלקה חדשה. רשת עמוקה לא יכולה להתאמן על זה. אבל אפשר להשתמש ברשת קיימת לחילוץ מאפיינים. ואז לאמן SVM על המאפיינים האלו.

זו גישה פופולרית ב-Meta-Learning.

6.4 יישומים מודרניים: Modern Applications

6.4.1 רפואה: Medicine

SVM נפוץ מאוד ברפואה [6]. כמה דוגמאות:

- אבחון סרטן מתמונות רנטגן

- חיזוי תגובה לתרופות

- סיווג סוגי תאים מנתוני גנום

הסיבה: דאטאסטים קטנים ודרישות פרשנות.

6.4.2 פיננסים: Finance

בנקים משתמשים ב-SVM לזיהוי הונאות. חברות ביטוח משתמשות בו להערכת סיכונים. בתי השקעות משתמשים בו לחיזוי שווקים. הסיבה: דרישות רגולטוריות לשקיפות.

6.4.3 ביואינפורמטיקה: Bioinformatics

סיווג רצפי חלבונים. זיהוי גנים. חיזוי מבנה מולקולרי. הסיבה: נתונים יקרים ומועטים, צורך בפרשנות מדעית.

6.5 מה למדנו?: What Did We Learn?

הבה נסכם את המסע.

פרק 0: הקדמה. למדנו את ההיסטוריה. איך SVM צמח מתוך חורף ה-AI. למה הוא רלוונטי גם היום.

פרק 1: אופטימיזציה. הבנו מה זה תכנות ליניארי וריבועי. למדנו שבעיות קמורות הן "טובות" – יש להן פתרון יחיד.

פרק 2: גיאומטריה. גילינו שמכפלה סקלרית היא כלל החלטה. הבנו מה זה היפר-מישור ושוליים גיאומטריים. הכרנו את וקטורי התמיכה.

פרק 3: מנוע האופטימיזציה. ראינו את הניסוח הנאיבי הבעייתי. למדנו שלושה טריקים שהופכים אותו לבעיית QP. הגענו לניסוח הסופי האלגנטי.

פרק 4: יישום. כתבנו קוד Python. השתמשנו ב-cvxopt ו-sklearn. למדנו סיווג רב-מחלקתי.

6.6 **לסיים: Conclusion**

SVM הוא אלגוריתם יפה. לא כי הוא הכי חזק. לא כי הוא הכי מודרני. אלא כי הוא שקוף, מובן, ואמין.

בעולם של קופסאות שחורות, זה יתרון. בעולם של דרישות רגולטוריות, זה הכרח. בעולם של נתונים מועטים, זה הצלה.

SVM לא ימות. הוא פשוט ימצא את המקום שלו. לצד רשתות עמוקות, לא במקומן. כחלק מארגז הכלים, לא במקום כולו.

זה השיעור האחרון. הכלים הטובים ביותר הם אלו שאנחנו מבינים. SVM הוא כזה.

6.7 English References

- 1 M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969, ISBN: 978-0-262-63022-1.
- 2 F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, 386–408, 1958. DOI: [10 . 1037/h0042519](https://doi.org/10.1037/h0042519)
- 3 D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, 533–536, 1986. DOI: [10 . 1038 / 323533a0](https://doi.org/10.1038/323533a0)
- 4 V. Vapnik and A. Lerner, “Pattern recognition using generalized portrait method,” *Automation and Remote Control*, vol. 24, 774–780, 1963.
- 5 C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, 273–297, 1995. DOI: [10 . 1023/A:1022627411411](https://doi.org/10.1023/A:1022627411411)
- 6 A. Ben-Hur, C. S. Ong, S. Sonnenburg, B. Schölkopf, and G. Rätsch, “Support vector machines and kernels for computational biology,” *PLoS Computational Biology*, vol. 4, no. 10, e1000173, 2008. DOI: [10 . 1371/journal.pcbi.1000173](https://doi.org/10.1371/journal.pcbi.1000173)
- 7 G. B. Dantzig, *Linear Programming and Extensions*. Princeton University Press, 1963, ISBN: 978-0-691-08000-0.
- 8 V. Klee and G. J. Minty, “How good is the simplex algorithm?,” 159–175, 1972.
- 9 N. Karmarkar, “A new polynomial-time algorithm for linear programming,” *Combinatorica*, vol. 4, no. 4, 373–395, 1984. DOI: [10 . 1007/BF02579150](https://doi.org/10.1007/BF02579150)
- 10 S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004, ISBN: 978-0-521-83378-3. [Online]. Available: <https://web.stanford.edu/~boyd/cvxbook/>
- 11 M. S. Andersen, J. Dahl, and L. Vandenberghe, *CVXOPT: Python software for convex optimization*, Version 1.2.5, 2020. [Online]. Available: <https://cvxopt.org/>
- 12 F. Pedregosa et al., “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, 2825–2830, 2011.
- 13 C.-C. Chang and C.-J. Lin, “LIBSVM: A library for support vector machines,” *ACM Transactions on Intelligent Systems and Technology*, vol. 2, no. 3, 27:1–27:27, 2011. DOI: [10 . 1145/1961189.1961199](https://doi.org/10.1145/1961189.1961199)
- 14 Y. Tang, *Deep learning using linear support vector machines*, Workshop contribution to ICML 2013 Challenges in Representation Learning, 2013. arXiv: [1306 . 0239 \[cs.LG\]](https://arxiv.org/abs/1306.0239).